Virtual memory for LLM agent context: the xCABI standard and the ctxvm engine.

Abstract

The context window of a large language model (LLM) is not memory. It is a small, fast, expensive cache. Every tool result, every system prompt, and every stale message stays in that window for the whole session. The result is waste: the model pays for tokens it never reads.

This paper describes `ctxvm`, a virtual-memory manager for LLM agent context. It has three parts:

1. **xCABI** — an open standard for agent traces. It records pages, renders, evictions, and faults in a deterministic log. Any consumer can replay any turn and get the same bytes.
2. **ctxvm-core** — a Rust engine. It keeps a page table, renders turns deterministically, and replays traces. It never mutates messages. It only observes.
3. **The measurement stack** — a trace corpus, a metric simulator, and policy leaderboards. These are packaged as open data products (ODPS 4.1).

Version 0 is a read-only flight recorder. You add it to any agent loop. Nothing changes. You get a trace, touch statistics, and waste numbers. Paging control comes later, behind an opt-in policy flag.

This paper explains what the framework is, how to use it, and where it stops.

Context tracing is not logging — it is the ability to reconstruct, verify, and replay the exact input an agent received. Every other improvement — token savings, quality, safety, compliance — starts from knowing what the model actually saw. That knowledge is necessary for each of them and sufficient for none: a faithful record of the input does not by itself evaluate a model, reach its reasoning, or make it behave.

---

1. The problem

LLM agents run in turns. Each turn sends a message array to the model. That array grows forever: tool results, long prompts, chat history, images. The model context window is fixed in size. When it fills, the system must drop or compress old content.

Today, every team builds its own solution. One team truncates the middle of the chat. Another summarizes old turns. Another sends the whole history every time. These formats are private. The logs are private. The policies are private. You cannot move a trace from one harness to another. You cannot replay a turn to see what the model saw. You cannot measure waste in the same way across two agents.

This is the pre-SQL state of agent memory. The mechanism works. The shared layer does not exist.

1.1 The cost of waste

Measured data shows the scale of the problem:

- Across 857 production sessions and 4.45 billion effective input tokens, 21.8% of tokens are structural waste. The waste breaks down as: unused tool schemas 11.0%, duplicated content 2.2%, stale tool results 8.7%. Stale results are reprocessed at a median amplification of 84.4×. (Source: Mason, arXiv:2603.09023; the system is named Pichay. Verified from the primary source, whose body states 4.45 billion effective input tokens while its abstract states 4.45 million — this paper cites the body.)
- Pichay reduces context size from 5,038 KB to 339 KB (93% reduction) over 681 production turns, with a fault rate of 0.0254% across 1.4 million simulated evictions. (Source: Mason, arXiv:2603.09023, verified.)

Waste costs money and latency. Every resident token is recomputed every turn. Tokens the model never reads still consume prompt cost and attention budget.

1.2 What is missing

No public standard exists for:

- a deterministic agent-trace format,
- a cross-harness replay of any turn,
- a conformance test for trace engines,
- a public corpus of paging traces for policy research.

ctxvm fills this gap. It is the gauge, the dyno, and the SQL for the context layer — not another pager.

2. Background and prior work

Operating systems solved this problem decades ago. Virtual memory gives a program the illusion of unlimited RAM. Only the pages the program touches are resident. A reference to a missing page raises a page fault. The OS loads that page from disk. Working-set theory keeps the resident set small. Belady's algorithm evicts the page needed furthest in the future.

Researchers now apply the same ideas to LLM context. The following prior work is verified.

2.1 Mason's Pichay — demand paging for LLM context (verified)

Pichay is a transparent proxy between a client and an inference API. It treats the context window as L1 cache. It evicts stale content, leaves a retrieval handle, detects a page fault when the model re-requests evicted content, and pins working-set pages from fault history.

Key results (verified from arXiv 2603.09023):

- Offline replay across 1.4 million simulated evictions: fault rate 0.0254%.
- Live production over 681 turns: context reduced up to 93% (5,038 KB to 339 KB).
- Under extreme sustained pressure, the system thrashes: repeated fault-in of evicted content.

Pichay shows that OS memory abstractions apply directly to LLM context windows. It also shows that models understand retrieval handles — they can re-request evicted content without fine-tuning.

The arXiv ID is 2603.09023 (v1, March 2026). Author: Tony Mason (University of British Columbia / Georgia Institute of Technology). The paper is self-published on arXiv; I could not confirm a conference venue. Treat any venue claim as UNVERIFIED.

2.2 Neural Paging — learned page controllers (verified)

Neural Paging (arXiv 2603.02228, Chen & Liu, February 2026) defines the Context Paging Problem. It proposes a learned Page Controller that approximates "Semantic Belady's Optimality" — retaining tokens with high future utility.

Key results (verified from arXiv 2603.02228):

- A Hierarchical Neural Turing Machine separates reasoning (the LLM) from memory management (the Page Controller).
- Under bounded window size K, it reduces long-horizon reasoning complexity from O(N^2) to O(N · K^2).
- It provides a robustness bound (Theorem 4) for policy-dependent access.
- Validation on synthetic paging traces confirms the bounds hold.

Neural Paging shows that learned paging policies are a viable research direction. The paper's own scope note says end-to-end evaluation on real LLM agents is outside its scope. That gap — no public trace corpus, replay simulator, or benchmark for learned paging policies — is what ctxvm's measurement stack targets.

2.3 Virtual Context — OS-style virtualization (verified)

Virtual Context (virtual-context.com) virtualizes LLM context the way an OS virtualizes memory. A client addresses 20 million tokens. The real window is 200K. The system compresses by topic, extracts structured facts, and pages content in and out on demand.

Key features (verified from virtual-context.com/paper/):

- Three-layer hierarchy with bidirectional paging (raw turns, segment summaries, tag summaries).
- Five retrieval tools: `find_quote`, `query_facts`, `expand_topic`, `remember_when`, `recall_all`.
- LRU eviction manages the token budget.

Term origin: the phrase "virtual context management" traces to MemGPT (arXiv 2310.08560, Packer et al., 2023), which coined it in the OS-for-LLMs sense and led to the Letta project. The Virtual Context system described here (virtual-context.com, Kidwai) is a separate, later implementation of the same idea.

2.4 What ctxvm adds

ctxvm does not invent a new paging algorithm. It builds the shared layer that prior work lacks:

1. A deterministic trace standard (xCABI RFC-0 with strict JSON Schema).
2. A Rust engine that replays any trace byte-identically.

3. A metric simulator with four formulas: net token reduction, fault recall, touch ratio, thrash index.
4. Packaging as ODPS 4.1 open data products, so corpora and leaderboards are public and verifiable.

Version 0 observes only. Version 1 adds paging control behind an opt-in policy flag.

---

3. What ctxvm / xCABI is

ctxvm is a virtual-memory manager for LLM agent context. It ships as three layers: a standard, an engine, and a measurement stack. The standard defines the format. The engine implements it. The measurement stack makes the results usable for research and operations.

3.1 The three pillars

The plan defines three pillars. Each addresses a gap in current agent-context practice.

Pillar 1 — xCABI, a portable trace standard. Every agent system keeps a private mutation log. None is cross-harness replayable, diffable, or conformance-testable. xCABI ("eXchangeable Context ABI") fills that gap. It defines a deterministic log of pages, renders, evictions, and faults. A consumer reconstructs any turn without knowing which harness produced the trace.

Pillar 2 — Lazy handles at the tool boundary. Current systems intercept waste after tools return bytes. xCABI returns typed page handles instead of payloads. This fixes re-mention amplification before waste enters the stream. Mason measured stale results reprocessed at a median amplification of 84.4× (verified from arXiv 2603.09023, Section 1). ctxvm does not implement paging control in v0. It builds the measurement surface that makes paging control measurable later.

Pillar 3 — The measure: corpus, simulator, conformance, leaderboard. Learned paging controllers (e.g. Neural Paging, arXiv 2603.02228) lack a public trace corpus, replay simulator, or benchmark. ctxvm provides all four: a trace corpus in ODPS 4.1 packaging, a metric simulator, a conformance validator, and a policy leaderboard scraped from manifest KPIs.

3.2 Strategy: telemetry-first, control-later

The crowded field builds control-plane pagers that fight for defaults. ctxvm escapes via the OpenTelemetry playbook:

- v0 is a read-only flight recorder. You wrap any agent loop. Nothing changes. You get the trace, touch statistics, and waste numbers. The plan estimates that agents pay for most rendered tokens they never read (an internal estimate, not an external measurement).
- Every install contributes traces to the corpus organically.
- Adoption is a config step, not a rewrite.
- Paging control ships in v1 behind an opt-in `policy` flag.

3.3 Core definitions

Term	Meaning
Page	A unit of context with stable identity. A page always has a `page_id`, a `kind` (file, tool_result, summary, image_ref), and a token count.
page_id	Stable page identity: a pair `(logical_ref, content_hash)`. Example: `tool:read_file(path=src/auth.ts)` plus the SHA-256 of the file content.
Trace	An append-only sequence of xCABI events, one JSON object per line (JSONL).
Turn	The determinism unit. All events for one model call belong to one turn.
Render	The message array the model actually sees for a turn. Full content for resident pages, stub text for evicted pages.
Fault	A reference to evicted content. The model re-requests it (tool_recall), or a scan finds an identifier for it (identifier_scan), or the harness requests it explicitly (explicit_api).
Stub	The inline placeholder the model sees after a page is evicted. It identifies what left, how to recover it, and costs less than the page.

3.4 The xCABI event vocabulary

The draft event vocabulary (from `02-xcabi-events-v0.md`) defines eight event types plus session metadata:

Event	Payload	Purpose
`session.open`	agent id, model, provider, budget, tokenizer id	Namespaces a trace
`page.declare`	page descriptor	A page enters the pager
`page.render`	turn #, mode (full or stub)	Ground truth for waste measurement
`page.evict`	turn #, stub text used	The paging decision and what the model was told
`page.fault`	turn #, source, latency	The economics event — a fault measures an eviction mistake
`page.pin` / `page.unpin`	reason	Working-set decisions become auditable
`turn.render`	turn #, resident/evicted token totals, headroom	The render set for that turn
`call.meta`	provider, estimated cache hit/miss, latency	Cache-cost honesty; estimates are flagged as estimates

Derived events (`trace.gc`, waste statistics) stay out of the log. The profiler computes them from `page.render` and `page.fault` at read time. The log remains a pure determinism surface.

3.5 Log invariants

xCABI imposes five invariants that make the standard meaningful:

1. Append-only. Turns are the determinism unit.
2. Every re-render of the same trace yields the identical message array.
3. Evictions batch at turn boundaries, never mid-turn. Transcript shape stays stable through stubs.

4. Different-harness consumers replay any turn exactly.
5. Every cost figure in derived analytics subtracts estimated cache misses (`call.meta.cacheEstimate`). Net cost is the only honest number.

3.6 Fault taxonomy

xCABI classifies faults by source:

Source	Mechanism	Default
`tool_recall`	Model re-calls the same tool with the same args → fault-in (Pichay-proven)	on
`identifier_scan`	Scan model output for paths, keys, URLs, symbols → fault-in	on
`explicit_api`	Harness or plugin calls `vm.fault(h)`	on
`semantic_*`	Embedding or LLM guesses ("it probably wants this")	v1 plugin only, marked non-deterministic

Semantic faults are explicitly excluded from the core. Core determinism means no LLM or embedding consult happens inside `ctxvm-core`. Smart decisions are plugins, logged as marked non-deterministic event classes.

3.7 Stub format

When a page is evicted, the model sees an inline stub with identity and self-service recovery. No instructional prompt pollution is added:

```
[Paged out: read src/auth/session.ts (8.2k tok, 187 lines).
 Re-call read_file(path="src/auth/session.ts") to re-materialize.]
```

The stub performs three functions: identify what left, how to recover it, cost less than the page. Pichay shows models understand this untaught. Hints escalate only if fault recall suffers.

3.8 The ctxvm-core engine

`ctxvm-core` is a Rust crate with no heavy dependencies. It implements:

- A stateful page table keyed by `(logical_ref, content_hash)`.
- A deterministic renderer: a pure function of (events, indices). Two renders of the same trace produce byte-identical output.
- A replay engine: read JSONL events, re-apply the renderer, compare against the expected final render totals. Divergence exits with code 1 and reports a structured error.
- Conformance vectors: golden trace files under `ctxvm-conformance/vectors/`. Every event type round-trips byte-identically through the codec.

M0 (the current milestone) is complete: 28+ tests green, 0 warnings, replay determinism byte-identical, conformance validator 108/108 PASS.

Bindings are planned but not yet built: napi-rs for TypeScript/Node, PyO3/maturin for Python. Adapters (OpenCode, Vercel AI SDK, LangGraph, MCP shim) live above the bindings as thin layers. None of this code exists yet; `design/mcp-shim.md` and `design/adapters.md` contain the integration plans.

3.9 The measurement stack

The measurement stack makes ctxvm usable for research:

- **Metric simulator** (`benchmarks/metrics/metrics.py`): pure Python functions over JSONL event streams implementing four formulas from `07-benchmarks-go-no-go.md`:
- `net_token_reduction = 1 - (paged_net_tokens / naive_tokens)`
- `fault_recall = recovered_faults / evictions`
- `touch_ratio = touched_tokens / rendered_tokens` (waste = 1 - touch_ratio)
- `thrash_index = mean(faults_per_turn)` over sustained-pressure windows
- **Reference implementation**: 18/18 pytest green, two synthetic fixtures (healthy and thrashing), CLI prints JSON.
- **ODPS 4.1 packaging**: the trace corpus is an open data product (`04-odps-dataproduct-corpus.yaml`), validated `valid`, `compliance_level: full`, `production_ready: true` against SDK 0.4.0. An ODCS v3.0.2 data contract (`contracts/corpus-0.1.odcs.yaml`) lints green via datacontract-cli 1.2.7.
- **Hub**: `hub/` contains an ODPC v1.0 catalog (`catalog.yaml`) and static HTML (`index.html`), built byte-identically by `tools/build-hub.sh`. hub.ctxvm.dev is a Git repo, not a bespoke server.

3.10 Locked decisions (what ctxvm is not)

These decisions are fixed and must not be revisited without orchestrator approval:

- Rust core + napi-rs / PyO3 bindings.
- Telemetry-first v0: the tracer never mutates messages.
- Determinism boundary: no LLM or embedding in core; semantic faults are plugins, marked non-deterministic.
- Zero prompt pollution in v0: identity stubs carry recovery instructions inline.
- Evictions batch at turn boundaries. `call.meta` carries estimated cache cost.
- ODPS / ODCS strictly at the edge: never in the Rust core.
- Corpus data quality maps replay-determinism → `validity` (ODPS) / `conformity` (ODCS).
- Vanilla names (`ctxvm`, `xCABI`) are working names. Renaming is an explicit orchestrator task.

3.11 What context tracing actually is

The one-liner: Context tracing is not logging — it is the ability to reconstruct, verify, and replay the exact input an agent received.

Conventional observability tells you what the agent did. It records decisions: which tools were called, what arguments were passed, what errors occurred. This is necessary but not sufficient. When an agent produces a wrong answer, a harmful action, or an unexpected result, the question is not "what did the agent do?" but "what did the agent see when it did it?"

The gap between "did" and "saw" is where most agent bugs live. Consider two scenarios:

1. **The model had the information and ignored it.** The file contents were resident in context. The model read them and made the wrong decision anyway. This is a model-quality problem.
2. **The model never saw the information.** The file contents were evicted (stubbed) or never declared. The model made a decision without the relevant context. This is a context-management problem.

Conventional logging cannot distinguish these two cases. It records the tool call but not the context that surrounded it. Context tracing records both: the tool call *and* the page table state, the resident/stubbed token totals, and the exact rendered message array.

What context tracing enables:

1. **Deterministic replay.** The Rust renderer is a pure function of (events, indices). Two renders of the same trace produce byte-identical output. You can reproduce the exact input the model received at any turn, independent of model variance, network latency, or rate limits.
2. **Session handoff without loss.** The trace up to turn N is a complete snapshot. A new session resumes from the snapshot with byte-identical context state. Resident pages appear as full content; evicted pages appear as stubs with recovery instructions.
3. **Agent forensics.** When an agent fails, the trace reveals not just what it did but what it saw. The fault taxonomy (tool_recall, identifier_scan, explicit_api) reveals when the model re-requested content it already had — a signal of confusion, not just a cost metric.
4. **Quality assurance and regression testing.** Traces enable deterministic CI: record a trace from a known-good run, replay it, verify that the agent's context matches. This catches context bugs — pages that should be resident but are stubbed, pages that should be stubbed but are resident — that conventional tests miss.
5. **Capacity planning.** Per-turn, per-page token accounting reveals which pages consumed the most tokens, which turns had the highest context pressure, and where the budget went. The KPI formulas (touch ratio, fault recall, net token reduction, thrash index) answer "can I reduce the budget?" without guessing.
6. **Compliance and audit.** The trace is a checksummed record of what the model saw: checksums prove the bytes are unchanged since they were written, which is evidence of accidental corruption and tamper-evidence only against parties without write access to the file. A threat model where the traced agent can reach its own log needs something stronger — hash-chained events or a signed root — and that is not built. The replay engine verifies byte-identical re-render (exit code 0). The `skipped_untraced` counter detects when context was rendered without being declared — a fidelity gap that would be a compliance finding.

Why this matters for ctxvm specifically:

ctxvm's tracer already records all of this. The `page.render` event captures the exact render mode (full or stub) for every page, every turn. The `turn.render` event captures resident/evicted token totals and headroom. The `page.fault` event captures when the model re-requested evicted content. The `call.meta` event captures estimated cache hit/miss and latency.

The gap is not in recording — it is in *using* the record. The trace enables deterministic replay, but the replay engine is a verification tool, not an action tool. The trace enables session handoff, but the handoff is within-harness only. The trace enables compliance, but the compliance check requires running `ctxvm replay` manually.

The path from "recording" to "acting" is the control beat (see §7, Limitations): turning the trace into eviction decisions, stub replacements, and budget enforcement. But the recording comes first — you cannot act on what you cannot measure, and you cannot measure what you do not record.

The one-liner restated: Every other improvement — token savings, quality, safety, compliance — starts from knowing what the model actually saw. Knowing it is necessary for each of them and sufficient for none. Context tracing supplies that, and only that; the improvements have to be built on it. ctxvm's tracer already records it; the remaining work is turning the trace into action.

---

4. How to use ctxvm

This section is a practitioner's guide. It covers the v0 tracer (read-only, no behavior change), the CLI, the xCABI trace format, and the adoption path. Nothing here assumes paging control; v0 observes only.

4.1 The v0 tracer (TypeScript)

The tracer is the only integration surface a harness needs. It is approximately ten lines of code:

```
import { trace } from "@ctxvm/node"

const vm = trace({
  agent: "my-agent",
  model: "openai/gpt-5",
  budgetTokens: 120_000,
})

// When tool results materialize, declare pages (auto-tokenized)
const h = vm.page({ kind: "tool_result", ref: "tool:read_file(path=src/auth.ts)",
tokens: n })

// Per turn: mirror what the model will actually see
vm.render(messages)                          // emits turn.render

// After the turn: scan model output for identifiers → record faults
vm.observe(modelOutputText)                  // source: identifier_scan
```

The tracer never mutates `messages`. It derives and logs only. Adoption is a config step: wrap the agent loop, nothing inside it changes.

An optional auto-shim wraps MCP tool calls: results are auto-declared as pages, and `tool_recall` faults fire by name+args matching (per Pichay). One-line swap from `client.callTool(...)` to `vm.call(...)`.

4.2 Tracer API surface

Method	Purpose
`trace(opts)`	Create a VM instance: agent id, model, provider, budget, tokenizer id
`vm.page(desc)`	Declare a page when a tool result materializes; returns a handle
`vm.render(messages)`	Emit `turn.render` — mirror of what the model sees this turn
`vm.observe(text)`	Scan model output for identifiers; record faults via `identifier_scan`
`vm.scan_text(text)`	Standalone identifier scan over arbitrary text
`vm.observe_recall(tool, args)`	Record a `tool_recall` fault when the model re-calls a tool
`vm.fault(handle)`	Explicit fault-in request from the harness

Errors (dangling pins, unclaimed faults, divergent replays) are logged as events, never thrown. Telemetry-first means the tracer cannot harm the host agent.

4.3 CLI surface

```
ctxvm profile ./session.jsonl
# Pages: 312        Rendered tokens: 41.8k
# Never-touched: 65.3% of rendered tokens (27.3k)
# Worst page: tool:read_file(src/db/schema.sql) 12.1k tokens — 0 touches
# Faults: 19 (all recovered via tool_recall) → eviction mistakes cost 0.4% net tokens

ctxvm replay ./session.jsonl      # byte-identical re-render; exit 1 on divergence
ctxvm diff a.jsonl b.jsonl        # page-level divergence of two runs
ctxvm export --odps ./session.jsonl   # dataproduct.yaml + llms.txt (telemetry → packaging)
```

`ctxvm profile` is the demo moment: it turns a trace into waste numbers. `ctxvm replay` enforces the core invariant — same trace, byte-identical render, exit 1 on divergence. `ctxvm diff` compares two traces at page granularity. `ctxvm export --odps` packages a session as an ODPS data product (manifest + llms.txt + checksum).

4.4 Tracer-first adoption path

The plan's strategy is telemetry-first, control-later. A team adopts ctxvm in three steps:

1. **Add the tracer.** Wrap the agent loop with `trace()`. No message changes, no prompt pollution. You get a trace immediately.
2. **Measure.** Run `ctxvm profile` on recorded sessions. See touch ratio, never-touched pages, fault rate, net token cost. Establish a baseline.
3. **Page (v1, optional).** Behind an opt-in `policy` flag, enable eviction and pinning. The same trace format and replay engine validate the pager: replay must remain byte-identical, and faults must stay below the target.

At no point does the harness rewrite its message assembly. The tracer mirrors; it does not edit.

4.5 The xCABI trace format

A trace is append-only JSONL: one JSON object per line. Every event carries a turn number, so turns are the determinism unit. A minimal trace looks like:

```
{"type":"session.open","agent":"my-agent","model":"openai/gpt-5","pro-
vider":"openai","budgetTokens":120000,"tokenizer":"tiktoken"}
{"type":"page.declare","pageId":["tool:read_file(path=src/
auth.ts)","sha256:abc123"],"kind":"tool_result","tokens":8192}
{"type":"page.render","turn":1,"mode":"full"}
{"type":"turn.render","turn":1,"residentTokens":8192,"evictedTokens":0,"headroom":111808}
{"type":"page.evict","turn":1,"stubText":"[Paged out: read src/auth/session.ts (8.2k
tok, 187 lines). Re-call read_file(path=\"src/auth/session.ts\") to re-materialize.]"}
{"type":"page.fault","turn":2,"source":"tool_recall","latencyMs":45}
{"type":"call.meta","turn":2,"provider":"openai","cacheEstimate":
{"hit":0.8,"miss":0.2}}
```

The strict JSON Schema (`spec/rfc-0/x-cabi.schema-v0.json`) enforces `additionalProperties: false` at event level while tolerating `x-` extensions. The conformance validator (`spec/rfc-0/validate_schema.py`) runs 108 checks over all fixtures: 108/108 PASS on M0.

4.6 What you can do with a trace

- **Replay any turn byte-identically.** The Rust renderer is a pure function of (events, indices). Two renders of the same trace produce identical output.
- **Diff two traces.** Page-level comparison shows where two agent sessions diverged.
- **Package as an ODPS data product.** `ctxvm export --odps` emits a manifest, llms.txt, and checksum. The manifest carries DQ checks (replay determinism, pin lint, redaction scan) and KPIs (fault recall, net token reduction, touch ratio).
- **Audit.** Evictions batch at turn boundaries; every cost figure subtracts estimated cache misses. Net cost is the only honest number.

4.7 Where v0 stops

v0 observes only. It does not evict, pin, or page. It does not consult an LLM or embedding. Semantic faults (`semantic_*`) are v1 plugins, marked non-deterministic, and excluded from the core. If you need paging control, wait for v1 behind the opt-in `policy` flag.

---

5. Architecture

ctxvm is a Rust workspace with four crates. The architecture maps 1:1 to the plan (§3 of `01-PLAN.md`).

Crate	Responsibility
`ctxvm-core`	Page table, budget, policy traits, deterministic renderer, replay engine. Zero heavy dependencies.
`ctxvm-codec`	xCABI serde structures + JSONL (`serde`/`serde_json` live here).
`ctxvm-tokenizer`	Token counting backends behind feature flags (tiktoken default; HF optional). Stub only in M0.
`ctxvm-conformance`	Golden vectors + replay conformance checks. Executable truth of the spec.

5.1 Data flow

1. The harness declares pages as tool results materialize (`page.declare`).
2. Per turn, `vm.render(messages)` emits `turn.render` — the mirror of what the model sees.
3. After the turn, `vm.observe(modelOutputText)` scans for identifiers and records faults (`identifier_scan`).
4. Events are appended to a JSONL trace. The trace is the only artifact.
5. `ctxvm replay` re-applies the renderer to the trace and compares against expected render totals. Divergence exits 1.

5.2 Determinism boundary

Core = pure replayable engine. No LLM, no embedding, no network. Semantic or embedding-based faulting lives in policy plugins, logged as marked non-deterministic event classes. This is a hard boundary, not a preference: it is what makes replay byte-identical and what lets a consumer verify any trace without trusting a model.

5.3 Bindings and adapters (planned, not built)

Bindings sit above `ctxvm-core`: napi-rs for TypeScript/Node, PyO3/maturin for Python. Adapters (OpenCode plugin, Vercel AI SDK middleware, LangGraph, MCP shim) live above the bindings as thin glue. The T7 survey pins anchors at OpenCode commit `e965774f` and vercel/ai commit `36e4ce7c`; each adapter is estimated at ≤50 lines of glue. None of this code exists yet — `design/mcp-shim.md` and `design/adapters.md` contain the plans.

5.4 Packaging boundary

ODPS / ODCS live strictly at the edge. The Rust core never knows what a manifest is. CLI and binding layers emit YAML. This keeps `ctxvm-core` small, dependency-free, and free of standards-format concerns.

---

6. Metrics

The metric simulator (`benchmarks/metrics/metrics.py`) implements four formulas over JSONL event streams, exactly as defined in `07-benchmarks-go-no-go.md`:

Metric	Formula	Meaning
Net token reduction	`1 - (paged_net_tokens / naive_tokens)`	How many tokens the pager saves vs naive stuffing
Fault recall	`recovered_faults / evictions`	Share of evictions the pager recovered from
Touch ratio	`touched_tokens / rendered_tokens`	Share of rendered tokens the model actually read; waste = 1 - touch ratio
Thrash index	`mean(faults_per_turn)` over sustained-pressure windows	Fault rate under pressure — thrashing detection

Cache-miss normalization is noted, not invented: net cost subtracts estimated cache misses (`call.meta.cacheEstimate`).

6.1 Reference implementation

Pure Python, 18/18 pytest green, two synthetic fixtures (healthy session, thrashing session), CLI prints JSON. Fixtures pass schema conformance 108/108 via the xCABI validator.

6.2 Where metrics go

Metric claims live in `productStrategy.productKPIs` of the ODPS manifest — machine-readable, scraped by the leaderboard. The leaderboard (v1) parses KPIs from manifests: fault recall, net token reduction, touch ratio. Non-deterministic policies must submit variance over ≥3 seeds.

---

7. Limitations

These are honest boundaries of the current design. They are not bugs; they are scope decisions.

- **v0 observes only.** No eviction, pinning, or paging. If you need the pager, wait for v1 behind the opt-in `policy` flag.
- **No semantic faults in core.** `semantic_*` (embedding/LLM guesses) are v1 plugins, marked non-deterministic. Core determinism requires this exclusion.
- **No bindings yet.** napi-rs (TS) and PyO3 (Py) are planned, not built. The tracer API in §4 is the target surface, not shipping code.
- **No adapters yet.** OpenCode and Vercel AI SDK integration plans exist (`design/adapters.md`); no code. The T7 survey anchors at specific commits but no dev build has run.
- **Fault recall target is unproven.** The M0 gate requires ≥95% fault recall (identifier_scan + tool_recall layered), but this is a target, not a measured result on real sessions.
- **Net token reduction target is unproven.** ≥1.5× at equal accuracy is the M0 gate; no real-session measurement exists yet.
- **Names are working names.** `ctxvm` and `xCABI` are not final. Renaming is an explicit orchestrator task.
- **Cache-hit estimates are flagged, not authoritative.** `call.meta` carries estimated hit/miss; providers report where available, otherwise the estimate is flagged as such.

---

8. Related work

Section 2 covers Pichay (arXiv 2603.09023), Neural Paging (arXiv 2603.02228), and Virtual Context (virtual-context.com) — all verified from primary sources. This section adds context on where ctxvm fits in the broader landscape.

8.1 MemGPT

MemGPT (Packer et al., 2023, arXiv 2310.08560) drew the OS analogy for LLM context: main context + external context, with the LLM issuing system calls to manage memory. Neural Paging critiques this design: the LLM acts as both kernel and application, spend-

ing tokens on housekeeping. ctxvm agrees — the tracer is a flight recorder, not a kernel; the LLM never manages its own memory.

8.2 LLMLingua and context compression

LLMLingua (Jiang et al., 2024) removes low-information tokens from prompts. ctxvm takes a different approach: structural removal of entire waste categories (unused tool schemas, duplicates, stale results) without lossy compression of remaining content. Pichay makes the same distinction.

8.3 SWE-Pruner

SWE-Pruner (Wang et al., 2026, arXiv 2601.16746) trains a neural skimmer for task-aware context pruning, achieving 23–54% token reduction on SWE-bench. This is a learned, model-dependent approach. ctxvm is orthogonal: it provides the trace corpus and replay simulator that learned pruning systems need for evaluation, but does not itself train or prune.

8.4 l1-pager (LangGraph)

`l1-pager` (github.com/sarath-m-s/l1-pager, v1.1.1 on PyPI and npm) is a Context Garbage Collector for LangGraph agents by Sarath M S (Feb–Mar 2026). It evicts large or old messages to a heap and restores them via a sentinel `fetch_evicted_page(page_id)` tool, with LRU / IMPORTANCE / HYBRID eviction policies. It is a real demand-paging-style system for LangGraph, but small-scale: roughly 5★ and ~200 weekly downloads on PyPI. It should be cited as prior art, not as ecosystem weight.

8.5 IBM chuk-ai-session-manager

`IBM/chuk-ai-session-manager` (GitHub, IBM-org OSS) is session management for AI applications: token monitoring, tool-call logging, and "infinite context" via automatic summarization. It is a session/context manager built on summarization, **not** a demand-paging system: it evicts by summarizing rather than by paging out with retrieval handles and fault-in. This distinction matters — summarization loses detail; paging preserves it behind a handle.

8.6 Why ctxvm is different

Pichay builds a pager. Neural Paging builds a learned controller. Virtual Context builds a full virtualization kernel. l1-pager builds a LangGraph garbage collector. IBM's session manager builds a summarization-based session manager. ctxvm builds the shared layer none of them provides: a deterministic trace standard, a replayable engine, a metric simulator, and ODPS-packaged corpora. It is the gauge and the dyno — not another pager.

---

9. Conclusion

LLM agent context is in the overlay era: every system manually assembles context on every API call, with no eviction policy, no demand loading, no working set, and no shared trace format. The mechanism works; the shared layer does not exist.

ctxvm is that shared layer. xCABI defines a deterministic, append-only, cross-harness trace format. `ctxvm-core` is a Rust engine that replays any trace byte-identically. The measurement stack — simulator, corpus, leaderboard — is packaged as ODPS 4.1 open data products.

Version 0 is a read-only flight recorder. Add it to any agent loop; nothing changes. You get traces, touch statistics, and waste numbers. Paging control ships in v1 behind an opt-in `policy` flag, validated by the same replay engine that guarantees byte-identical determinism.

Context tracing is not logging — it is the ability to reconstruct, verify, and replay the exact input an agent received. Every other improvement depends on knowing what the model actually saw. ctxvm's tracer already records this; the remaining work is turning the trace into action.

The field does not need another private pager. It needs a gauge, a dyno, and a SQL for the context layer. That is what ctxvm is built to be.

References

1. Mason, T. "The Missing Memory Hierarchy: Demand Paging for LLM Context Windows." arXiv:2603.09023 [cs.OS], March 2026. https://arxiv.org/html/2603.09023 — the system is named Pichay; verified from primary source.
2. Chen, L. and Liu, Q. "Neural Paging: Learning Context Management Policies for Turing-Complete Agents." arXiv:2603.02228 [cs.LG], February 2026. https://arxiv.org/html/2603.02228 — verified from primary source.
3. Kidwai, Y. A. "Virtual Context: Unbounded Context for LLM Agents via OS-Style Memory Management." virtual-context.com/paper/ — verified from primary source.
4. Packer, C. et al. "MemGPT: towards LLMs as operating systems." arXiv:2310.08560, 2023. — source of the "virtual context management" term; led to Letta.
5. Sarath M S. "l1-pager — Context Garbage Collector for LangGraph." github.com/sarath-m-s/l1-pager; PyPI l1-pager v1.1.1, 2026 — verified from primary source.
6. IBM. "chuk-ai-session-manager." github.com/IBM/chuk-ai-session-manager, 2026 — verified from primary source. Summarization-based session manager, not demand paging.
7. Jiang, Y. et al. "LLMLingua." 2024.
8. Wang, Y. et al. "SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents." arXiv:2601.16746, 2026.

Document control

Item	Value
Working title	Virtual memory for LLM agent context: the xCABI standard and the ctxvm engine
Format	Markdown single source (MyST for website, Markdown → LaTeX/WeasyPrint for PDF)
Style	ASD-STE100-inspired (active voice, defined terms, no idioms, no marketing language)
Audience	Coders, scientists, generally computer-literate readers
Phases complete	

Item	Value
	P1 (§1–2), P2 (§3), P3 (§4), P4 (§5–9 + references), P5 (final draft + PDF + web-site)
External claims	All web-verified from primary sources or marked UNVERIFIED
UNVERIFIED (not cited)	Pichay conference venue (arXiv only, no venue confirmed)